```
  client.open method, src
  client.send()

get = (src, callback) ->
  http "GET", src, callback
```

Unfortunately, there's a catch with using `XMLHttpRequest`. For security reasons, it works only if the data you're loading is on the same domain as your browser-based application. Suppose that in your case the dashboard is on one particular domain at http://www.agtronsemporium.com, and the data server is on a different domain at http://www.agtronsapi.com. How do you load data from a different domain?

11.2.2. Dynamic script insertion

An alternative technique to `XMLHttpRequest` that works across different domains is to take advantage of the browser's ability to dynamically load and execute new `<script>`s at any time. You might call this *hot code loading* because program code is being loaded while the program is running (while it's *hot*). To load an external script dynamically into the application, you dynamically insert a `script` element with the desired `src` attribute:

```
window.serverStatusCallback = (status) ->
  console.log status

head = document.querySelector 'head'
script = document.createElement 'script'
script.src = 'http://www.agtronsapi.com/server-status.js'
head.appendChild script
```

If the `server-status` script that's loaded has the data wrapped in the appropriate callback, as follows, then the status will be logged to the console:

```
window.serverStatusCallback({
  'server 1': {
    'cpu': 22,
    'network': {
      'in': 2343,
      'out' 3344
    }
```